

xHC: Expanded Hyper-Connections

TECHNICAL REPORT

Xiangdong Zhang^{1,2} Xiaohan Qin^{1,2} Sunan Zou^{2,4} Tuo Dai² Xiaoming Shi²
Huaijin Wu^{1,2} Yebin Yang^{1,2} Zhuo Xia^{1,2} Shaofeng Zhang³ Lin Yao²
Yuliang Liu⁵ Yu Cheng⁶ Junchi Yan^{1*}

¹School of AI, Shanghai Jiao Tong University ²Dots Studio, Xiaohongshu Inc.
³University of Science and Technology of China ⁴School of CS, Peking University
⁵Independent Researcher ⁶The Chinese University of Hong Kong

Abstract

Hyper-Connections (HC) expand the residual stream of Transformers into N parallel streams, providing a form of memory scaling beyond model width and depth. Manifold-Constrained HC (mHC) stabilizes this formulation at scale. The large gains from $N=1$ to $N=4$ suggest residual-stream expansion as a promising scaling axis. However, existing HC-family methods typically stop at $N=4$. Our experiments reveal why: scaling mHC beyond this point yields diminishing performance gains and rapidly increasing training cost. We attribute this limitation to two bottlenecks: insufficient write-back information for an expanding number of streams and residual-mixing generation whose cost scales cubically with N . To address both bottlenecks, we propose **xHC** (Expanded **H**yper-**C**onnections), the first HC-family method to achieve meaningful expansion beyond $N=4$. xHC combines temporal feature augmentation for richer write-back with a sparse residual-stream architecture that updates only $k=4$ of the $N=16$ streams while retaining dense access to the full residual state. **Across 18B and 28B MoE models, xHC delivers strong and consistent downstream improvements.** On an 18B MoE model, xHC improves the average downstream score by 4.0 points over mHC, while adding only modest training FLOPs over the vanilla baseline. Scaling-law experiments show that the vanilla and mHC baselines require $1.50\times$ and $1.19\times$ the compute of xHC, respectively, to reach the same loss. Practical large- N training also requires controlling memory traffic from the expanded residual state. We therefore introduce **xHC-Flash**, which reduces the per-sublayer memory traffic from $73.5C$ to $40C$, comparable to the $34C$ required by mHC at $N=4$, while retaining most of the gains of full xHC. Together, xHC and xHC-Flash make large- N residual-stream expansion effective and practical for LLM pre-training.

1 Introduction

Large language model architectures have advanced rapidly [28, 51, 14, 29, 43, 53] through improved attention mechanisms [46, 54, 34, 1], Mixture-of-Experts [10, 23], and scaling in width, depth, and data [25, 20]. Yet the residual stream that carries token representations across layers remains largely unchanged: it still operates as a single identity pathway, offering no learnable control over cross-layer information flow [17]. Hyper-Connections (HC) [55] challenge this design by replacing the single stream with N parallel residual streams governed by learnable mixing matrices. Building on HC, Manifold-Constrained HC

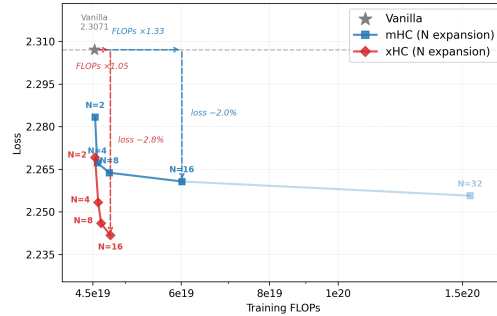


Figure 1: Expansion efficiency: loss vs. FLOPs on a 2.5B MoE model (details in Table 6).

*Corresponding author: yanjunchi@sjtu.edu.cn. Project page: <https://github.com/aHapBean/xHC>.

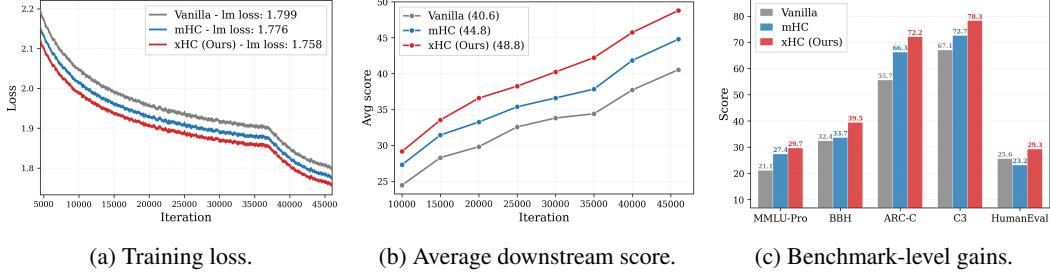


Figure 2: **xHC delivers broad gains at 18B scale.** (a) xHC achieves lowest training loss. (b) The loss improvement translates into a significantly higher average downstream score across benchmarks in Table 1. (c) xHC improves representative benchmarks across reasoning, knowledge, and code.

(mHC) [50] makes multi-stream residual training stable at scale and represents the state-of-the-art variant. Residual-stream expansion increases the persistent state of models with modest additional FLOPs, offering a form of *memory scaling* beyond width and depth. The strong gains observed when expanding from $N=1$ to $N=4$ in prior results [55] suggest that this is a promising scaling axis. However, existing HC-family methods [55, 50, 52, 31] typically stop at $N=4$. We study this through mHC, the state-of-the-art HC formulation at scale. Our experiments in Figure 1 show that naively increasing N further yields rapidly diminishing returns: loss improves only marginally from $N=4$ to $N=16$, while training FLOPs increase by 32%, leaving this scaling axis largely unrealized.

We argue that this benefit-cost imbalance is not incidental, but stems from two distinct bottlenecks: one limits the benefit of larger N , while the other increases its cost. The first is an **information bottleneck**: each stream is meant to store a different weighted history of layer outputs, but each layer injects only a single write-back signal into the multi-stream state. As N grows, forming meaningful and diverse stream histories requires richer write-back information than this single signal can provide, making additional streams increasingly redundant. The second is a **computational bottleneck**: in mHC, the dominant cost comes from generating the residual mapping $\mathcal{H}_{\text{res}} \in \mathbb{R}^{N \times N}$ through an input-dependent projection. Because this projection predicts N^2 coefficients from an NC -dimensional residual state, its cost scales as $O(N^3C)$, making the cost of expansion grow much faster than its performance benefit. Together, these bottlenecks explain why simply adding more streams does not translate into proportionally more useful capacity. They also clarify what it would take to make expansion rate a true scaling axis: the model must supply more diverse write-back information while avoiding the cubic cost of scaling $\mathcal{H}_{\text{res}}$.

We therefore propose **xHC** (Expanded Hyper-Connections), the first method to achieve *meaningful* expansion of HC-family models beyond $N=4$. Building on the stable multi-stream formulation of mHC, xHC makes large- N expansion both effective and affordable through two complementary designs. To increase the benefit of larger N , xHC introduces temporal feature augmentation along the causal token sequence: it enriches the write-back signal with low-cost features from neighboring tokens, using multi-scale causal convolutions to supply more diverse information as the number of streams grows. To control the cost of larger N , xHC introduces a sparse residual-stream architecture that activates only k out of N streams for residual mixing and write-back, while keeping the read path dense so that every layer can still access the full N -stream state. These two designs are structurally decoupled yet highly synergistic: as N grows, temporal augmentation becomes more useful because additional streams require richer write-back information, while sparse residual updates become more valuable because they substantially reduce the extra FLOPs introduced by large- N expansion.

Empirically, xHC delivers substantial gains over both mHC and the vanilla residual baseline. As shown in Figure 2, xHC consistently improves both pre-training loss and downstream performance at the 18B MoE scale. It reaches a lower final training loss than mHC and the vanilla baseline (1.758 vs. 1.776 and 1.799), and improves the average downstream score from 44.8 with mHC to 48.8, while adding only 4.1% training FLOPs relative to the vanilla baseline. xHC also remains effective when the backbone is trained with Muon, indicating that its gains are not specific to AdamW.

More importantly, xHC changes the benefit-cost tradeoff of scaling the expansion rate itself. As shown in Figure 1, on a 2.5B MoE model, increasing N from 4 to 16 in mHC reduces loss by only 0.006 while increasing training FLOPs by 32%. In contrast, increasing N from 4 to 16 in xHC reduces loss by 0.012 with only 4% extra FLOPs. This shows that xHC makes larger N substantially more cost-effective, turning residual-stream expansion into a practical scaling axis.

This improved benefit-cost tradeoff also translates into better compute efficiency across model scales. Our scaling-law experiments in Figure 4 show that, to reach the same loss, the vanilla requires $1.50\times$ compute of xHC and mHC requires $1.19\times$ compute of xHC. This confirms that the improvement is systematic rather than confined to a specific model size.

Together, these results establish that xHC improves both the utility and FLOPs efficiency of residual-stream expansion. Practical large- N training, however, also requires controlling the memory traffic introduced by the expanded residual state. We therefore introduce xHC-Flash, which shares routing and dense read computation across sublayers to avoid repeated full-state accesses. It reduces the estimated per-sublayer traffic from $73.5C$ to $40C$, comparable to the $34C$ required by mHC at $N=4$, while retaining nearly all of the performance gains of full xHC. We further develop a fused implementation that consolidates residual-stream operations, reducing memory traffic and kernel launch overhead. **Our contributions are as follows:**

- 1) Bottleneck diagnosis.** We identify two bottlenecks behind the diminishing returns of larger N in mHC: limited write-back information supply and the cubic cost of dense residual mixing.
- 2) Expanded Hyper-Connections.** We propose the first HC-family method to achieve meaningful expansion beyond $N=4$, through two coordinated designs: temporal feature augmentation for richer write-back, and sparse residual updates for affordable residual mixing.
- 3) Large-scale validation.** We validate xHC on 18B and 28B MoE models and observe strong and consistent downstream improvements across both scales. Scaling laws and N -sweeps further demonstrate systematic compute and expansion efficiency, while experiments with Muon confirm that xHC remains effective beyond AdamW.
- 4) Efficient deployment.** We introduce xHC-Flash and fused kernel implementations to reduce memory traffic and kernel launch overhead in large- N residual-stream expansion. xHC-Flash ($N = 16, k = 4$) achieves per-sublayer memory traffic comparable to mHC at $N=4$ ($40C$ vs. $34C$) while retaining most of the performance gains of full xHC.

2 Related Work

Cross-Layer Information Flow. Residual connections [17] are the default mechanism for carrying token representations across layers in Transformer language models [46]. Most LLM architectures [29, 14, 51] keep this residual state as a single stream, while improving its optimization or weighting through gates, learned scaling, or depth-dependent rescaling [38, 45, 3, 47]. Other methods improve cross-layer access or feature reuse [21, 33, 44, 56], but operate over previous activations rather than expanding the persistent residual state. Closest to our work, Hyper-Connections (HC) [55] expand the residual stream into N learnable streams, and Manifold-Constrained HC (mHC) [50] stabilizes this formulation at scale via Sinkhorn-normalized residual mixing, achieving the strongest performance among HC-family methods. Other HC variants [52, 31] improve expressivity or memory efficiency, but existing HC-family methods largely operate at $N=4$, leaving open whether residual-stream expansion can become a true scaling axis for LLM pre-training. xHC bridges this gap by making larger- N expansion both meaningful and affordable.

Sparse Computation in Language Models. Sparsity is widely used to scale language models efficiently. Mixture-of-Experts (MoE) applies sparsity over computation: each token is routed to a subset of experts, decoupling total parameters from per-token FLOPs [36, 26, 10, 29, 43]. Sparse attention instead sparsifies over tokens, restricting each query to attend to a subset of the sequence and reducing long-context attention cost [6, 4, 54, 29]. xHC applies sparsity on a different dimension: *residual streams*. It activates only k out of N streams for residual mixing and write-back. This reduces the dominant residual-mixing cost from scaling with all N streams to scaling with only the k active streams, making large- N residual-memory expansion affordable.

3 Method

We first recap the HC formulation and its manifold-constrained variant (§3.1), then analyze why scaling the expansion rate saturates in existing designs (§3.2). We then present xHC (§3.3; Figure 3), which uses two coordinated designs to make large- N expansion effective and affordable: temporal feature augmentation enriches the write-back signal (§3.3.1), while a sparse residual-stream architecture reduces the cost of large- N residual mixing (§3.3.2).

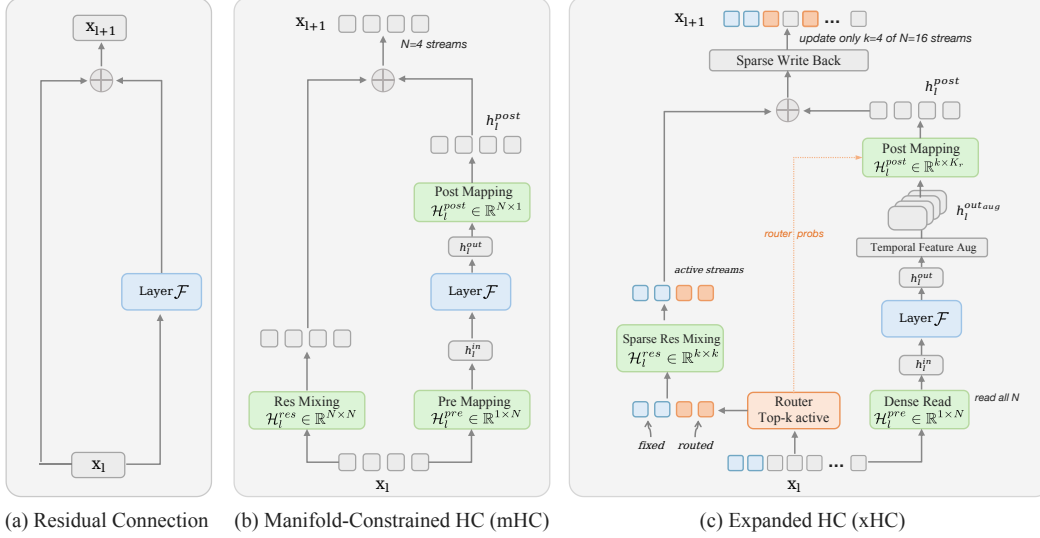


Figure 3: **Overview of xHC.** (a) A standard Transformer layer maintains a single residual stream. (b) mHC expands the residual state into $N=4$ streams with dense residual mixing and write-back. (c) xHC scales to $N=16$ with only $k=4$ active streams: it reads all streams, applies the sublayer $\mathcal{F}$ (Attn/MLP), augments MLP outputs, and sparsely writes back to selected streams. Blue/orange streams denote fixed/routed active streams. Restricting residual mapping to the k active streams reduces the dominant residual-mapping generation cost from $O(N^3C)$ to $O(k^3C)$.

3.1 Preliminaries

Hyper-Connections. Standard residual connections [17] maintain a single residual stream across layers, limiting the capacity of cross-layer information flow. Hyper-Connections (HC) [55] generalize this design by maintaining N parallel residual streams, yielding significant pre-training improvements with modest additional FLOPs. Let $X_l = (x_{l,1}, \dots, x_{l,N})^\top \in \mathbb{R}^{N \times C}$ denote the multi-stream state at layer l , where C is the hidden dimension. HC introduces three learnable mappings per layer: a *pre-mapping* $\mathcal{H}_l^{\text{pre}} \in \mathbb{R}^{1 \times N}$ that aggregates streams into a single input, a *post-mapping* $\mathcal{H}_l^{\text{post}} \in \mathbb{R}^{N \times 1}$ that distributes the layer output back onto streams, and a *residual mapping* $\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{N \times N}$ that mixes streams. The single-layer update is:

$$X_{l+1} = \mathcal{H}_l^{\text{res}} X_l + \mathcal{H}_l^{\text{post}} \mathcal{F}(\mathcal{H}_l^{\text{pre}} X_l, \mathcal{W}_l), \quad (1)$$

where $\mathcal{F}(\cdot, \mathcal{W}_l)$ is the layer function (Attention or MLP) with parameters $\mathcal{W}_l$. In the dynamic variant, all three mappings are predicted from the current state via lightweight projections gated by tanh.

Manifold-Constrained HC. When $\mathcal{H}_l^{\text{res}}$ is unconstrained, the composed mapping $\prod_l \mathcal{H}_l^{\text{res}}$ across depth can amplify or attenuate signals, breaking the identity mapping property that keeps deep residual training stable. mHC [50] addresses this by projecting $\mathcal{H}_l^{\text{res}}$ onto the Birkhoff polytope of doubly stochastic matrices via the Sinkhorn–Knopp algorithm:

$$\mathcal{H}_l^{\text{res}} = \text{SK}(\exp(\alpha_l^{\text{res}} \cdot (x'_l W_l^{\text{res}}) + b_l^{\text{res}})) \in \mathbb{R}^{N \times N}, \quad (2)$$

where $x'_l \in \mathbb{R}^{NC}$ is the normalized flattened multi-stream state, $W_l^{\text{res}} \in \mathbb{R}^{NC \times N^2}$ predicts the residual-mixing logits, α_l^{res} and b_l^{res} are learnable scale and bias, and $\text{SK}(\cdot)$ denotes Sinkhorn normalization [37] that enforces row and column sums to one. This constraint preserves stable information propagation across depth, enabling practical large-scale deployment at $N=4$ with significant performance gains.

3.2 Why Scaling N Saturates in mHC

Understanding this saturation is essential for turning residual-stream expansion from a small- N improvement into a practical scaling axis. Prior results [55] show that early residual-stream expansion is highly effective: increasing N from 1 to 4 brings substantial gains at modest FLOPs cost (less than 2%). This suggests that scaling N could improve model performance by increasing residual memory,

providing an axis orthogonal to width and depth. However, existing HC-family methods [55, 52, 31, 50] typically stop at $N=4$, leaving open whether this axis remains meaningful at larger N . To answer this, we sweep $N \in \{2, 4, 8, 16, 32\}$ in mHC under matched training recipes. As shown in Figure 1, scaling beyond $N=4$ quickly runs into diminishing returns: increasing N from 4 to 16 reduces loss by only 0.006, while training FLOPs increase by 32%. This unfavorable benefit-cost tradeoff motivates us to examine what prevents larger N from translating into useful capacity. We identify two bottlenecks behind this saturation.

Information Supply Bottleneck. In mHC, each of the N streams accumulates a distinct weighted combination of historical layer outputs. However, the write-back to stream i at layer l takes the form:

$$\Delta x_{l,i} = h_{l,i}^{\text{post}} \cdot \text{out}, \quad (3)$$

where $h_{l,i}^{\text{post}}$ is a scalar and $\text{out} \in \mathbb{R}^C$ is the layer output. Although $h_{l,i}^{\text{post}}$ is input-dependent and stream-specific, the newly injected information at each layer is spanned by only one write-back component, out : different streams can assign different weights to this component, but cannot draw from different components. This can be sufficient at small N , but as N grows, additional streams need more diverse write-back components to form non-redundant histories. Without such diversity, extra streams become increasingly redundant. Section 4.5 provides evidence for this bottleneck.

Cost Bottleneck. The second bottleneck is on the cost side. In mHC, generating $\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{N \times N}$ from the NC -dimensional state requires predicting N^2 coefficients, leading to an $O(N^3C)$ input-dependent projection cost. This makes large- N expansion increasingly expensive even when the additional performance gain is marginal. Concretely, mHC at $N=16$ adds roughly 32% extra FLOPs relative to its $N=4$ setting for only limited loss reduction on 2.5B MoE, as shown in Figure 1.

The two bottlenecks compound: the information bottleneck limits the *benefit* of larger N , while the cost bottleneck increases its *price*. Together they collapse the return on investment of scaling N .

3.3 xHC: Expanded Hyper-Connections

xHC addresses both bottlenecks to make expansion rate N a practical scaling axis (Figure 3). It enriches write-back signals with local contextual features and adopts an asymmetric sparse residual-stream architecture: only k active streams undergo residual mixing and write-back, while dense read keeps every layer connected to all N streams. This reduces the dominant $\mathcal{H}_l^{\text{res}}$ generation cost from $O(N^3C)$ to $O(k^3C)$, with $k=4$ and $N=16$ in our main setting. Other implementation details follow mHC unless stated otherwise.

3.3.1 Enriching the Write-Back Signal

Temporal Feature Augmentation. As analyzed in §3.2, large- N expansion saturates because each layer provides only a single write-back component to all streams. A direct fix would be to compute a separate output for each stream, but this would multiply the layer FLOPs by N . Instead, xHC enriches the write-back basis by borrowing low-cost local contextual information from neighboring tokens. Because autoregressive prediction already conditions on surrounding context, neighboring hidden outputs are semantically compatible with the current token and can be integrated into its write-back signal. We aggregate this local information with lightweight causal depthwise 1D convolutions, which preserve autoregressive order and add small overhead [16, 49, 15].

To describe local convolution, we temporarily restore the sequence dimension and write $\text{out} \in \mathbb{R}^{S \times C}$, where S is the sequence length. We apply r causal depthwise convolutions with kernel sizes $\{\kappa_1, \dots, \kappa_r\}$ to the layer output. Different kernel sizes capture neighboring-token information at different contextual ranges, providing multi-granularity write-back components that expand the write-back basis beyond a single layer output. We concatenate these components with the original output:

$$\text{out}_{\text{aug}} = [\text{out}; \text{DWConv}_{\kappa_1}(\text{out}); \dots; \text{DWConv}_{\kappa_r}(\text{out})] \in \mathbb{R}^{S \times K_r \times C}, \quad (4)$$

where $K_r = r + 1$. In our main setting, $r = 3$ with kernel sizes $\{4, 8, 12\}$, so $K_r = 4$. The temporal convolution branches are zero-initialized, preserving the original write-back path at initialization while allowing contextual components to be learned during training. The convolutions are per-channel and causal, adding only $C \sum_{j=1}^r \kappa_j$ parameters per layer. We apply temporal augmentation only after MLP (MoE FFN) layers: attention already mixes positions, and adding temporal augmentation after attention empirically destabilizes training.

Gram–Schmidt Orthogonalization. Because depthwise convolutions operate channel-wise, their outputs can retain a strong component aligned with the original layer output. If these correlated components are directly combined by $\mathcal{H}_l^{\text{post}}$, the augmented write-back signal may contain redundant components and amplify the original direction in an uncontrolled manner, making write-back scaling less predictable at large N . We therefore apply modified Gram–Schmidt orthogonalization [13] over the K_r components. Let $g_j = \text{DWConv}_{\kappa_j}(\text{out})$ denote the j -th convolutional component, and initialize $v_1 = \text{out}$. For $j = 1, \dots, r$, we compute:

$$v_{j+1} = g_j - \sum_{i=1}^j \frac{\langle g_j, v_i \rangle}{\langle v_i, v_i \rangle} v_i. \quad (5)$$

After orthogonalization, we redefine $\text{out}_{\text{aug}} = [v_1; \dots; v_{K_r}] \in \mathbb{R}^{S \times K_r \times C}$ and use these orthogonalized components for all subsequent write-back operations. The projection is computed per token along the C -dimensional space and adds small overhead.

3.3.2 Sparse Residual-Stream Architecture

The cost bottleneck comes from residual mixing over all N streams. xHC reduces this cost by updating only k active streams out of N , while keeping dense read access to the full N -stream state. We describe the forward pass in three steps: routing, reading, and writing.

Stream Routing. A router selects the k streams to update at each sublayer. Let $\tilde{x}_l \in \mathbb{R}^{NC}$ denote the LayerNorm-normalized [2] flattened multi-stream state used by the router. The router observes the full N -stream state and produces per-stream importance scores:

$$s = \sigma(\tilde{x}_l W_r) \in \mathbb{R}^N, \quad (6)$$

where $W_r \in \mathbb{R}^{NC \times N}$ is a learned projection and σ is the sigmoid function. We use sigmoid rather than softmax to reduce winner-take-all routing. For stability, we use a fixed-plus-routed scheme: m streams are always active with routing weight 1, while the remaining $k - m$ active streams are selected by TopK routing over the non-fixed streams [26, 36]. Let $\mathcal{I} = (\mathcal{I}_1, \dots, \mathcal{I}_k)$ denote the selected active stream indices:

$$\mathcal{I} = \text{FixedIdx} \cup \text{TopK}_{i>m}(s_i, k - m), \quad p_j = \begin{cases} 1, & \mathcal{I}_j \in \text{FixedIdx}, \\ s_{\mathcal{I}_j}, & \mathcal{I}_j \notin \text{FixedIdx}, \end{cases} \quad j = 1, \dots, k. \quad (7)$$

Although the projection produces scores for all N streams, the scores of the fixed streams are ignored in TopK selection. This full-score projection makes it easier to fuse router generation with the pre-mapping projection (Section 5.3) while guaranteeing m persistent active paths.

Dense Read. A naive sparse variant would sparsify both reading and writing, but this can disconnect cross-layer information flow: streams updated at one layer may not be selected for reading by the next, shortening the effective path through which routed information propagates. xHC therefore keeps the read path dense:

$$\text{input}_l = \sum_{i=1}^N h_{l,i}^{\text{pre}} \cdot x_{l,i}, \quad \text{where } \mathcal{H}_l^{\text{pre}} = f_{\text{pre}}(X_l) \in \mathbb{R}^{1 \times N}. \quad (8)$$

Thus every layer can access the full N -stream state, preserving cross-layer information flow even though only k streams are updated. Ablations in Section 4.5 confirm the importance of dense read.

Sparse Residual Update. In mHC, $\mathcal{H}_l^{\text{res}}$ and $\mathcal{H}_l^{\text{post}}$ are generated from the full N -stream state. xHC instead applies the same generators only to the active state X_{active} :

$$\mathcal{H}_l^{\text{res}} = \text{SK}(f_{\text{res}}(X_{\text{active}})) \in \mathbb{R}^{k \times k}, \quad (9)$$

$$\mathcal{H}_l^{\text{post}} = f_{\text{post}}(X_{\text{active}}) \in \mathbb{R}^{k \times K_r}. \quad (10)$$

Here, f_{res} and f_{post} follow the residual and post-mapping generators of mHC, but operate on the active streams. The post-mapping $\mathcal{H}_l^{\text{post}}$ generalizes the mHC form from $\mathbb{R}^{N \times 1}$ to $\mathbb{R}^{k \times K_r}$, so that each active stream can independently combine the K_r augmented write-back components. This reduces the dominant residual-mapping generation cost from $O(N^3 C)$ to $O(k^3 C)$ while retaining the Sinkhorn constraint on the active stream subset.

Algorithm 1 Forward pass of one xHC sublayer

Require: $X \in \mathbb{R}^{N \times C}$ (multi-stream state), sublayer $\mathcal{F}$ (Attention/MLP)

```
1:
2: /* Stream routing */
3:  $s \leftarrow \sigma(\text{LN}(\text{flatten}(X)) W^r)$ ,  $\mathcal{H}^{\text{pre}} \leftarrow f_{\text{pre}}(X)$  //  $\mathbb{R}^N, \mathbb{R}^N$ 
4:  $\mathcal{I}, p \leftarrow \text{Route}(s; m, k)$  //  $m$  fixed and  $k-m$  routed streams
5:  $X_{\text{act}} \leftarrow \text{Gather}(X, \mathcal{I})$  //  $\mathbb{R}^{k \times C}$ 
6:
7: /* Mapping generation (from active streams) */
8:  $\mathcal{H}^{\text{res}}, \mathcal{H}^{\text{post}} \leftarrow f_{\text{res}}(X_{\text{act}}), f_{\text{post}}(X_{\text{act}})$  //  $\mathbb{R}^{k \times k}, \mathbb{R}^{k \times K_r}$ 
9:
10: /* Dense read  $\rightarrow$  sublayer  $\rightarrow$  temporal feature augmentation */
11:  $\text{out} \leftarrow \mathcal{F}\left(\sum_{i=1}^N \mathcal{H}_i^{\text{pre}} \cdot x_i\right)$  //  $\mathbb{R}^C$ 
12: if  $\mathcal{F}$  is MLP then
13:    $\text{out}_{\text{aug}} \leftarrow \text{TempAug}(\text{out})$  //  $\mathbb{R}^{K_r \times C}$ 
14: else
15:    $\text{out}_{\text{aug}} \leftarrow \text{out}$  //  $K_r = 1$ 
16: end if
17:
18: /* Sparse write-back */
19:  $X_{\text{act}}^{\text{new}} \leftarrow \mathcal{H}^{\text{res}} X_{\text{act}} + p \odot (\mathcal{H}^{\text{post}} \text{out}_{\text{aug}})$  //  $\mathbb{R}^{k \times C}$ 
20:  $X \leftarrow \text{Scatter}(X, \mathcal{I}, X_{\text{act}}^{\text{new}})$  // inactive streams unchanged
21: return  $X$ 
```

The active write-back signal is computed by contracting $\mathcal{H}_l^{\text{post}}$ with the augmented components:

$$\Delta X_{\text{active},j} = p_j \sum_{r=1}^{K_r} \mathcal{H}_{l,j,r}^{\text{post}} \text{out}_{\text{aug},r}, \quad j = 1, \dots, k, \quad (11)$$

where the token dimension is omitted for clarity. The active streams are then updated by:

$$X_{\text{active}}^{\text{new}} = \mathcal{H}_l^{\text{res}} X_{\text{active}} + \Delta X_{\text{active}}. \quad (12)$$

Here p_j is the routing weight for the j -th active stream: fixed streams use $p_j = 1$, while routed streams use their sigmoid scores. We apply p_j only to the new write-back term, leaving residual mixing ungated. Updated streams are scattered back to the full state, while non-active streams carry forward unchanged for future dense reads.

3.3.3 Why Both Designs Are Needed

The two designs address complementary bottlenecks. Temporal feature augmentation makes additional streams more informative, but alone leaves dense residual mixing expensive. Sparse residual updates make large N affordable, but alone leave the write-back signal information-limited. Together, they make large- N expansion both meaningful and affordable, with ablations in Section 4.5 confirming that neither design alone recovers the full benefit of xHC.

3.3.4 Parameterization Details

We now detail the concrete parameterization of the generators f_{pre} , f_{res} , and f_{post} left implicit in Eqs. 8–10. These follow a shared gated-projection pattern from mHC: a learnable gating scalar α (initialized to 0.01) scales the input-dependent term relative to a learnable bias, so that each mapping starts near its static initialization and gradually becomes dynamic. The stream router uses a direct linear projection followed by sigmoid activation (Eq. 6).

The pre-mapping is generated from the RMSNorm-normalized full state:

$$\mathcal{H}_l^{\text{pre}} = \sigma(\alpha^{\text{pre}} \cdot \text{RMSNorm}(\text{flatten}(X_l)) W^{\text{pre}} + b^{\text{pre}}), \quad W^{\text{pre}} \in \mathbb{R}^{N \times N}. \quad (13)$$

Table 1: Downstream evaluation on 18B and 28B MoE models. All numbers are scores (%; higher is better). mHC uses $N=4$, and xHC uses $N=16$, $k=4$; all methods use comparable training FLOPs.

Benchmark	18B			28B		
	Vanilla	mHC	xHC	Vanilla	mHC	xHC
<i>Language Understanding & Knowledge</i>						
MMLU	48.9	54.7	57.2	54.6	56.8	60.5
MMLU-Pro	21.1	27.4	29.7	30.1	34.9	36.0
MMLU-Redux	46.4	49.9	52.8	50.6	53.9	56.4
<i>Reasoning & Mathematics</i>						
BBH	32.4	33.7	39.5	41.7	43.6	43.4
CommonsenseQA	54.6	56.6	60.9	60.5	63.9	69.6
ARC-Challenge	55.7	66.3	72.2	70.8	74.9	77.7
GSM8K	37.7	44.5	48.4	50.3	56.3	59.2
<i>Code</i>						
HumanEval	25.6	23.2	29.3	27.4	26.8	31.1
LCBench	9.9	12.2	14.6	15.1	14.8	17.9
<i>Chinese</i>						
CMMLU	42.7	47.6	50.4	47.6	50.1	53.4
CEval	44.5	48.8	52.4	50.2	51.2	54.9
C3	67.1	72.7	78.3	75.2	78.7	82.5
Average	40.6	44.8	48.8	47.8	50.5	53.6

The residual and post-mapping generators use the RMSNorm-normalized active state $\tilde{x}_{\text{active}} = \text{RMSNorm}(\text{flatten}(X_{\text{active}})) \in \mathbb{R}^{kC}$:

$$\mathcal{H}_l^{\text{res}} = \text{SK}(\exp(\alpha^{\text{res}} \cdot \text{mat}_{k \times k}(\tilde{x}_{\text{active}} W^{\text{res}}) + b^{\text{res}})), \quad W^{\text{res}} \in \mathbb{R}^{kC \times k^2}, \quad (14)$$

$$\mathcal{H}_l^{\text{post}} = 2\sigma(\alpha^{\text{post}} \cdot \text{mat}_{k \times K_r}(\tilde{x}_{\text{active}} W^{\text{post}}) + b^{\text{post}}), \quad W^{\text{post}} \in \mathbb{R}^{kC \times kK_r}. \quad (15)$$

The $2\sigma(\cdot)$ scaling in the post-mapping allows coefficients in $[0, 2]$, enabling both attenuation and mild amplification of individual write-back components. The sigmoid in the pre-mapping constrains aggregation weights to $[0, 1]$. For the attention sublayer (no temporal augmentation), $K_r = 1$ and the post-mapping reduces to $\mathbb{R}^{k \times 1}$. The temporal convolution branches each have a kernel $w_j \in \mathbb{R}^{k_j \times C}$, computing $g_j[t] = \sum_{i=0}^{\kappa_j-1} w_j[i] \odot \text{out}[t-i]$ channel-wise; with $r=3$ branches of kernel sizes $\{4, 8, 12\}$, this adds only $24C$ parameters per MLP sublayer. Algorithm 1 summarizes the forward pass of a single xHC sublayer.

4 Experiments

We evaluate xHC through language model pre-training experiments across model scales, compute budgets, expansion rates, ablation settings, and optimizer choices. We first compare xHC against mHC and the vanilla residual baseline on 18B and 28B MoE models, then study compute efficiency through scaling-law experiments. Next, N -sweep and ablation studies examine whether larger expansion rates remain beneficial under xHC and whether the two designs address the large- N bottlenecks. Finally, we test compatibility with the Muon optimizer.

4.1 Experimental Setup

Training setup. We conduct experiments in the language model pre-training setting using Mixture-of-Experts (MoE) Transformer models. Unless otherwise stated, we apply xHC and mHC to the same DeepSeekMoE-style [10] Transformer backbone. Specifically, the backbone uses grouped-query attention (GQA) [1] and 144 experts with top-8 routing, where each expert is implemented as a SwiGLU-based FFN [35]. We report main downstream results at two MoE scales: an 18B-total, 1.7B-activated model and a 28B-total, 2.7B-activated model. We additionally use a 10B MoE model for ablations and a 2.5B MoE model for N -sweep experiments. Unless otherwise specified, xHC uses $N=16$ total streams with $k=4$ active streams. All models are trained with a context length of 8192 tokens on a data mixture covering English, Chinese, code, mathematics, and reasoning corpora. Unless otherwise specified, all experiments use AdamW [32] as the training optimizer. Full hyperparameters and training details are provided in Appendix A.

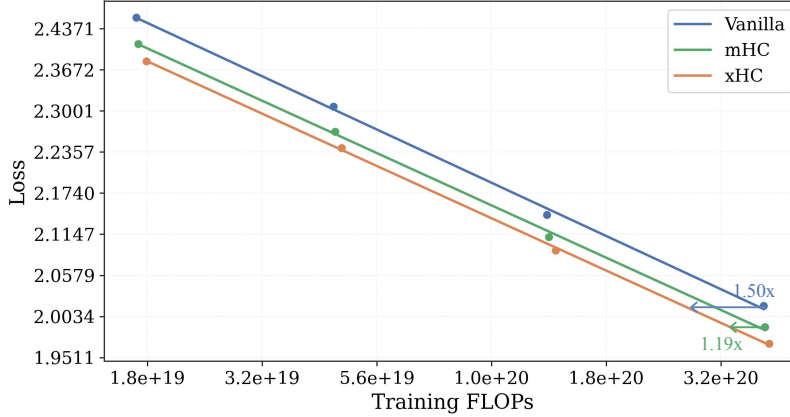


Figure 4: Scaling-law comparison. xHC traces a consistently lower loss curve than both mHC and the vanilla residual baseline across training compute.

Baselines. We compare xHC primarily against mHC ($N=4$) [50], which has comparable training FLOPs to xHC and serves as the strongest HC-family method at scale. We also include a standard residual-stream Transformer [46, 17] as the vanilla baseline.

Downstream Evaluation. We evaluate on a benchmark suite covering four aspects of LLM performance: language understanding and knowledge (MMLU [18], MMLU-Pro [48], MMLU-Redux [12]); reasoning and mathematics (BBH [41], CommonsenseQA [42], ARC-Challenge [7], GSM8K [8]); code generation and understanding (HumanEval [5], LCBench [9]); and Chinese language understanding (CMMLU [27], CEval [22], C3 [40]). Together, these benchmarks provide a compact yet diverse view of whether improved residual-stream scaling translates into stronger generalization after pre-training. Evaluation protocols and shot settings are provided in Appendix A.

4.2 Main Results

We compare xHC against mHC and the vanilla residual baseline on 18B and 28B MoE models. Table 1 summarizes downstream results under comparable training-FLOPs budgets, with xHC adding only 3.0% training FLOPs over the vanilla baseline at 28B.

xHC outperforms both mHC and the vanilla baseline across both scales. At 18B, the average score improves from 44.8 with mHC to 48.8 with xHC, with representative gains on ARC-Challenge (+5.9), BBH (+5.8), C3 (+5.6), and HumanEval (+6.1). At 28B, the average score improves from 50.5 to 53.6 (+3.1) over mHC, with representative gains on CommonsenseQA (+5.7), HumanEval (+4.3), C3 (+3.8), and MMLU (+3.7). The consistent improvements on the 28B MoE model further demonstrate that xHC remains effective in larger pre-training regimes.

4.3 Scaling Laws

The downstream results above establish the advantage of xHC at two discrete scales. We next ask whether this advantage persists across a broader compute range. To fit scaling laws [25], we train a separate suite of four models for each method under matched recipes, spanning training compute from 1.7×10^{19} to 4.0×10^{20} FLOPs. We fit a shifted power law [20, 19]:

$$\mathcal{L}(C) = AC^{-\alpha} + E, \quad (16)$$

where C denotes training FLOPs, A and α are fitted parameters, and $E = 0.72$ is the estimated irreducible loss. We fit by linear regression in log space and plot the resulting curves with the offset E included. Detailed settings are provided in Appendix B.

Figure 4 shows that xHC traces a lower fitted loss curve than both mHC and the vanilla baseline across the measured compute range. At the largest compute point, xHC reduces loss by about 1.1% relative to mHC and 2.4% relative to the vanilla baseline. To quantify this advantage, we use the losses achieved by the largest vanilla baseline and mHC models as representative targets, and read from the fitted xHC curve the compute required to match each target. The resulting matched-loss comparison shows that the baseline and mHC require about $1.50\times$ and $1.19\times$ the compute of xHC, respectively. This comparison, together with the consistently lower fitted curve, indicates that xHC

Table 2: Ablation study. Parentheses show extra training FLOPs over vanilla baseline. “Sparse” is the sparse architecture, “D. Read” is Dense Read, and “Fixed” is the number of always-active streams.

Method	N	Temp Aug.	Sparse	D. Read	k	Fixed	Router	Val. Loss ↓
(1) Vanilla baseline	-	-	-	-	-	-	-	2.029
(2) mHC (+0.6%)	4	-	-	-	-	-	-	2.004
(3) mHC (+18.8%)	16	-	-	-	-	-	-	1.998
(4) mHC w/ Temp Aug (+20.1%)	16	✓	-	-	-	-	-	1.984
(5) xHC (+3.3%)	16	✓	✓	✓	4	2	Sigmoid	1.983
(6) w/o D.Read & Fixed Streams	16	✓	✓	✗	4	0	Sigmoid	1.997
(7) w/o Dense Read	16	✓	✓	✗	4	2	Sigmoid	1.985
(8) w/o Fixed Streams	16	✓	✓	✓	4	0	Sigmoid	1.986
(9) $k=2$	16	✓	✓	✓	2	1	Sigmoid	1.991
(10) $k=8$	16	✓	✓	✓	8	2	Sigmoid	1.982
(11) Softmax Router	16	✓	✓	✓	4	2	Softmax	1.988

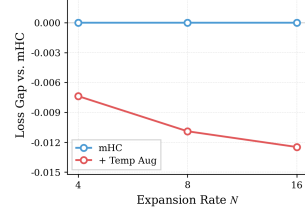


Figure 5: Information bottleneck ablation. The gain grows with N .

provides a systematic compute-efficiency improvement across the measured model family, rather than only improving specific downstream evaluation points.

4.4 Scaling the Expansion Rate

We evaluate whether xHC turns the expansion rate N into a meaningful scaling axis. We sweep $N \in \{2, 4, 8, 16\}$ in xHC on a 2.5B MoE model (details in Appendix A), and compare against mHC at matched expansion rates. In mHC, all N streams participate in dense residual mixing, so the residual-mixing overhead grows rapidly with N . In xHC, increasing N enlarges residual-memory capacity while the sparse update cost remains controlled by k . Figure 1 plots language modeling loss during training against training FLOPs as N increases. Under xHC, loss decreases consistently from $N=2$ to $N=16$, with each doubling of N yielding a clear improvement at small additional FLOPs. In contrast, mHC shows rapid saturation: increasing N from 4 to 16 reduces loss by only 0.006 while increasing training FLOPs by 32%. Over the same range, xHC reduces loss by 0.012 with only 4% additional FLOPs. These results show that xHC substantially improves the benefit-cost tradeoff of residual-stream expansion, making larger N useful under a practical compute budget.

4.5 Ablation Study

Ablation studies use a 10B MoE model trained under a matched data budget, providing a controlled setting for isolating the effect of each xHC design choice. Model configurations and training details are provided in Appendix A. We report validation loss on the Pile test set [11].

Incremental Construction. Following the motivation of xHC, we start from mHC at $N=16$ and incrementally add the two ingredients of xHC: temporal feature augmentation and a sparse residual-stream architecture. Temporal feature augmentation targets the information bottleneck by enriching write-back information, while sparse residual updates target the cost bottleneck by making large- N affordable. As shown in Table 2 (2)–(5), adding temporal feature augmentation to mHC improves validation loss from 1.998 to 1.984, confirming the benefit of enriching the write-back signal at large N . Adding the sparse residual-stream architecture then preserves this validation loss (1.983) while reducing FLOPs overhead from 20.1% to 3.3%, showing that sparse updates make the enriched large- N residual state affordable. Together, these results support the design of xHC: effective large- N expansion requires both richer write-back information and controlled residual-update cost.

Information Bottleneck Ablations. We add temporal feature augmentation alone to dense mHC at $N \in \{4, 8, 16\}$, using three multi-scale causal convolution branches by default. Figure 5 shows that the loss gap relative to mHC becomes increasingly negative as N grows. This trend is consistent with our diagnosis that the information bottleneck becomes more severe at larger expansion rates, where additional streams require richer write-back information to form non-redundant histories.

Sparse Architecture Ablations. Table 2 validates the designs that keep sparse residual updates effective. The central risk of sparse write is information disconnection: a stream updated at one layer may not be selected at the next, breaking cross-layer propagation. Removing both Dense Read and fixed streams exposes this risk, degrading loss to 1.997. Dense Read mitigates this by letting every layer access all N streams, so information in any stream remains visible regardless of routing; with fixed streams present, removing Dense Read increases loss from 1.983 to 1.985. Fixed streams further stabilize sparse updates by providing guaranteed write targets; removing them increases loss from 1.983 to 1.986. Separately, the active-stream budget k controls the sparsity–quality tradeoff: $k=2$ under-provisions active streams (1.991), $k=8$ yields a marginal gain (1.982) at higher cost, and

Table 3: Downstream evaluation on the 18B MoE model with Muon optimizer. “AdamW” and “Muon” columns are vanilla baselines under each optimizer; “Muon + xHC” applies xHC ($N=16$, $k=4$, without GS) on top of the Muon baseline.

Benchmark	AdamW (Tab. 1)	Muon	Muon + xHC
<i>Language Understanding & Knowledge</i>			
MMLU	48.9	51.3	56.6
MMLU-Pro	21.1	27.9	32.0
MMLU-Redux	46.4	49.0	52.2
<i>Reasoning & Mathematics</i>			
BBH	32.4	36.1	42.2
CommonsenseQA	54.6	49.1	61.3
ARC-Challenge	55.7	63.6	75.3
GSM8K	37.7	41.8	52.8
<i>Code</i>			
HumanEval	25.6	21.3	28.7
LCBench	9.9	11.0	16.0
<i>Chinese</i>			
CMMLU	42.7	45.2	50.2
CEval	44.5	47.2	54.3
C3	67.1	73.7	77.1
Average	40.6	43.1	49.9

$k=4$ balances the two. Finally, Sigmoid routing outperforms Softmax (1.983 vs. 1.988), consistent with avoiding winner-take-all routing that can leave streams persistently inactive.

4.6 Compatibility with Muon Optimizer

The experiments above use AdamW [32]. We further evaluate xHC under Muon [24, 30], which uses Newton–Schulz iteration to construct orthogonalized matrix updates and has shown strong results in Transformer pre-training [43]. This tests whether xHC remains effective beyond AdamW.

When training with Muon, we make two practical adaptations. **First, we remove Gram–Schmidt (GS) orthogonalization from temporal feature augmentation.** Under AdamW, GS acts as a stability mechanism: the causal convolution branches are often highly correlated with the original layer output, so without orthogonalization the post-mapping can combine near-parallel components and amplify the write-back signal along the same direction, leading to activation growth and gradient-scale spikes. Muon reduces this sensitivity by applying Newton–Schulz orthogonalization to backbone matrix updates, which keeps update spectra more controlled. Moreover, GS removes parallel components in the forward pass and correspondingly projects out gradient components along these directions; under Muon, we find this extra projection unnecessary and slightly restrictive. We therefore remove GS when training xHC with Muon. Second, we apply Muon only to the backbone attention and MLP projection matrices, while **keeping xHC-specific routing and mapping projections under AdamW.** These xHC projections are highly unbalanced matrices, mapping NC or kC hidden features to low-dimensional stream-control outputs such as N routing/pre-mapping weights, k^2 residual-mixing logits, and kK_r post-mapping weights. Under the default $N=16$, $k=4$ setting, their output dimensions are much smaller than their input dimensions, making them less suitable for Muon’s matrix orthogonalization. All other xHC components remain unchanged.

Table 3 reports results on the 18B MoE model under the same training recipe as Table 1, with Muon applied to backbone parameters and AdamW retained for xHC parameters; optimizer settings are provided in Appendix A. xHC maintains substantial gains over the Muon baseline across benchmarks, indicating that the architecture is compatible with Muon and remains effective beyond AdamW.

5 Practical Deployment

This section examines the practical training efficiency of xHC. Although xHC introduces modest additional FLOPs, maintaining multiple residual streams increases memory traffic and can affect

Table 4: Per-token memory access of residual-stream operations (sublayer function I/O excluded). Totals averaged per sublayer. *Once per block. †Includes MLP-side conv (+ C R, + $3C$ W) and Flash input correction (+ $2C$ R, + C W). ‡Amortized over 4 sublayers.

Operation	mHC		xHC ($N=16, k=4$)		xHC-Flash	
	R	W	R	W	R	W
Mapping generation	NC	N^2+2N	NC	$2N$	NC^*	$3N^*$
$\mathcal{H}^{\text{pre}}$ dense read	$NC+N$	C	$NC+N$	C	$(NC+2N)^*$	$2C^*$
Gather active streams	–	–	kC	kC	kC^*	kC^*
Active mapping generation	–	–	kC	k^2+kK_r	kC	k^2+kK_r
$\mathcal{H}^{\text{res}}$ residual mixing	$NC+N^2$	NC	$kC+k^2$	kC	$(kC+k^2)^*$	kC^*
$\mathcal{H}^{\text{post}}$ write-back	$C+N$	NC	K_rC+kK_r	kC	K_rC+kK_r	kC
Residual merge and scatter	$2NC$	NC	$2kC$	kC	$2kC$	kC
Per-sublayer, avg.†	$21C$	$13C$	$55C$	$18.5C$	$36C$	$15C$
xHC-Flash-4sub, avg.‡	–	–	–	–	$26.5C$	$13.5C$
Total I/O	$130C$ ($N=16$) $34C$ ($N=4$)		$73.5C$		$51C$ (xHC-Flash) $40C$ (xHC-Flash-4sub)	

wall-clock throughput. We first analyze its computation and memory-access overhead, then introduce xHC-Flash to reduce repeated full-state accesses. **Notably, xHC-Flash-4sub achieves memory traffic comparable to that of mHC at $N=4$ while maintaining a clear performance advantage.** Finally, we describe our infrastructure design, specifically the kernel-fusion strategy used to reduce the runtime overhead of the remaining operations.

5.1 Efficiency Analysis

The FLOPs overhead of xHC is modest: with our default $N=16, k=4$ setting, xHC adds 3.0% training FLOPs at the 28B scale. Detailed derivations of the training FLOPs and parameter overhead are provided in Appendix C. As identified in prior work [50], the primary source of runtime overhead for HC-family methods [55, 50] is often memory traffic rather than arithmetic FLOPs. Maintaining an N -stream residual state introduces additional I/O for normalization, projection, and residual mixing at every sublayer, which can directly affect training throughput.

Let C denote the model hidden dimension. Table 4 decomposes the per-token memory traffic of the residual-stream operations. Attention and MLP have asymmetric access costs because they use $K_r=1$ and $K_r=4$, respectively, and temporal augmentation is applied only to MLP. We therefore report amortized per-sublayer costs: the standard average divides the total traffic of one Attention/MLP block by two. With $N=16$ and $k=4$, xHC averages $55C$ reads and $18.5C$ writes per sublayer, for a total of $73.5C$. This is approximately $2.2\times$ the $34C$ required by mHC at its standard $N=4$ setting, primarily because xHC performs two full-state reads over the $NC=16C$ representation at each sublayer: one for mapping generation and one for dense-read aggregation. As an additional reference, mHC at the matched expansion rate of $N=16$ would require $130C$ per sublayer. Although sparse residual updates substantially reduce the cost of large- N expansion, xHC still incurs higher memory traffic than the mHC $N=4$ configuration, motivating the efficiency optimizations introduced next.

This gap motivates the two optimizations developed below: xHC-Flash reduces repeated full-state accesses at the architectural level, while our fused implementation reduces the overhead of the remaining residual-stream operations.

5.2 xHC-Flash: A Lightweight Variant

To reduce xHC’s memory traffic, we introduce **xHC-Flash**, which amortizes full-state operations across consecutive sublayers while preserving xHC’s performance advantage over mHC. Unless otherwise stated, xHC-Flash follows the full-xHC configuration: $N=16, k=4$, and MLP-side $K_r=4$ temporal augmentation. Sharing within one block reduces the per-sublayer traffic from $73.5C$ to $51C$. Its natural four-sublayer extension, **xHC-Flash-4sub**, further reduces the traffic to $40C$. At this level, its traffic is comparable to that of mHC at $N=4$ ($34C$), and it retains most of xHC’s performance gains (Table 4). Because only $k=4$ of $N=16$ streams change at each sublayer, xHC-Flash reuses

routing, jointly forms sublayer-specific pre-mappings and base readouts, and updates subsequent inputs through lightweight corrections instead of repeated full-state reads.

Key simplifications. xHC-Flash makes the following changes relative to the full xHC:

- **Shared routing:** a single routing decision computed from the block-entry state is shared by the Attention and MLP sublayers. This removes the MLP-side router projection and full-state normalization, at the cost of conditioning MLP routing on the pre-Attention state.
- **Joint pre-mapping with separate weights:** the Attention and MLP pre-mappings, $\mathcal{H}^{\text{pre,Attn}}$ and $\mathcal{H}^{\text{pre,MLP}}$, are generated jointly from the block-entry state using separate projection weights, producing separate base readouts:

$$\text{inp}_A = \sum_{i=1}^N \mathcal{H}_i^{\text{pre,Attn}} x_i, \quad \text{inp}_M = \sum_{i=1}^N \mathcal{H}_i^{\text{pre,MLP}} x_i. \quad (17)$$

The separate weights preserve sublayer-specific stream aggregation, which we find important for retaining model performance. Both pre-mappings are conditioned on the block-entry state. After the Attention write-back, inp_M is corrected using only the changes to the active streams (Eq. 19), avoiding a second full-state read. We refer to this exact correction as *dense-read reuse*.

- **Remove Attention-side $\mathcal{H}^{\text{res}}$:** to enable exact dense-read reuse without a second full-state read, we remove residual mixing from the Attention sublayer, yielding

$$X_{\text{act}}^{\text{new}} = X_{\text{act}} + p \odot (\mathcal{H}^{\text{post,Attn}} \text{out}_{\text{Attn}}), \quad (18)$$

where $\mathcal{H}^{\text{post,Attn}} \in \mathbb{R}^{k \times 1}$ is the Attention post-mapping ($K_r=1$). This also removes a residual-mapping projection with a $kC \times k^2$ weight matrix and one Sinkhorn normalization per block; active-stream mixing is deferred to the MLP-side $\mathcal{H}^{\text{res}}$.

Since each active stream receives only a scaled copy of the Attention output, the change to the precomputed MLP base readout is a token-wise scalar multiple of out_{Attn} :

$$\text{input}_{\text{MLP}} = \text{inp}_M + \alpha \text{out}_{\text{Attn}}, \quad \alpha = \sum_{j=1}^k \mathcal{H}_{T_j}^{\text{pre,MLP}} p_j \mathcal{H}_j^{\text{post,Attn}}. \quad (19)$$

Here, α is assembled from quantities already computed during the Attention update. Appendix E.1 provides the derivation.

Algorithm 2 summarizes how xHC-Flash shares full-state operations and updates active streams within one Transformer block.

Four-sublayer extension. The design extends naturally from two to four consecutive sublayers. In xHC-Flash-4sub, a single routing decision is shared across the two-block group, while four separate sublayer-specific pre-mappings, $\{\mathcal{H}^{\text{pre},q}\}_{q=1}^4$, and their corresponding non-active base readouts are jointly formed from the group-entry state. Unlike the two-sublayer case, later corrections must account for both Attention and MLP write-backs; the latter contains $K_r=4$ temporal branches and cannot always be reduced to a scalar multiple of a single output vector. We therefore use the active-stream state already carried through the four sublayers to form later inputs, instead of maintaining a separate delta buffer. The active-state read is fused with active-side mapping computation, so the input correction does not introduce an additional active-state pass and retains the $2C$ read and C write correction cost used in Table 4. Residual mixing is applied only at the final MLP, followed by a single scatter to the full state. Appendix E.2 gives the equivalent correction form and derivation.

Performance. Table 5 reports the accuracy-efficiency trade-off. xHC-Flash matches full xHC in validation loss (1.983) while reducing I/O from $73.5C$ to $51C$. xHC-Flash-4sub further reduces I/O to $40C$, comparable to the $34C$ of mHC at $N=4$, while maintaining a clear loss advantage over mHC (1.984 vs. 2.004). The small loss change suggests that sharing routing decisions and generating sublayer-specific pre-mappings from a short-window entry state does not significantly reduce the benefit of residual-stream expansion; Appendix E.3 provides a detailed discussion.

5.3 Infrastructure Design

A direct implementation of xHC consists of many small, largely memory-bound operators that repeatedly access the residual state, materialize normalized features and intermediate mappings, and

Algorithm 2 xHC-Flash: one Transformer block

Require: $X \in \mathbb{R}^{N \times C}$

```

1:
2: /* Joint pre-forward (once per block) */
3:  $s, \mathcal{H}_A^{\text{pre}}, \mathcal{H}_M^{\text{pre}} \leftarrow \text{JointFullStateMap}(\text{flatten}(X))$ 
4:  $\mathcal{I}, p \leftarrow \text{Route}(s; m, k); \quad X_{\text{act}} \leftarrow \text{Gather}(X, \mathcal{I})$  //  $k$  active streams
5:  $\text{inp}_A, \text{inp}_M \leftarrow \text{JointDenseRead}(X, \mathcal{H}_A^{\text{pre}}, \mathcal{H}_M^{\text{pre}})$ 
6:
7: /* Attention (no residual mixing) */
8:  $\mathcal{H}_A^{\text{post}} \leftarrow f_{\text{post}}^A(X_{\text{act}})$  //  $\mathbb{R}^{k \times 1}$ 
9:  $\text{out}_A \leftarrow \text{Attn}(\text{inp}_A)$ 
10:  $X_{\text{act}} \leftarrow X_{\text{act}} + p \odot (\mathcal{H}_A^{\text{post}} \text{out}_A)$ 
11:  $\alpha \leftarrow \sum_{j=1}^k \mathcal{H}_{M, \mathcal{I}_j}^{\text{pre}} p_j \mathcal{H}_{A, j}^{\text{post}}$ 
12:  $\text{input}_M \leftarrow \text{inp}_M + \alpha \text{out}_A$  // exact dense-read correction
13:
14: /* MLP (with residual mixing + temporal aug) */
15:  $\mathcal{H}^{\text{res}}, \mathcal{H}_M^{\text{post}} \leftarrow f_{\text{res}}(X_{\text{act}}), f_{\text{post}}^M(X_{\text{act}})$  //  $\mathbb{R}^{k \times k}, \mathbb{R}^{k \times K_r}$ 
16:  $\text{out}_M \leftarrow \text{MLP}(\text{input}_M)$ 
17:  $X_{\text{act}} \leftarrow \mathcal{H}^{\text{res}} X_{\text{act}} + p \odot (\mathcal{H}_M^{\text{post}} \text{TempAug}(\text{out}_M))$ 
18:
19:  $X \leftarrow \text{Scatter}(X, \mathcal{I}, X_{\text{act}})$ 
20: return  $X$ 

```

Table 5: xHC-Flash comparison on the 10B MoE model. I/O is the estimated memory traffic (read + write) amortized per sublayer from Table 4.

Method	Val. Loss ↓	I/O / sublayer
Vanilla baseline	2.029	$3C$
mHC ($N=4$)	2.004	$34C$
xHC ($N=16, k=4$)	1.983	$73.5C$
xHC-Flash	1.983	$51C$
xHC-Flash-4sub	1.984	$40C$

incur kernel-launch overhead. Our implementation targets the default $N=16$ and $k=4$ configuration and organizes these operations into two stages: mapping generation and mapping application. Within each stage, operations sharing the same inputs are fused to reduce memory traffic and kernel launches. Residual states are stored in bfloat16, and projection operands use bfloat16, while normalization statistics, routing, mapping coefficients and Sinkhorn iterations use float32.

Mapping generation. The first stage generates the router logits and the three xHC mappings, $\mathcal{H}^{\text{pre}}$, $\mathcal{H}^{\text{post}}$, and $\mathcal{H}^{\text{res}}$. The router and pre-mapping project the same flattened state $x \in \mathbb{R}^{NC}$ but use different normalizations: LayerNorm for routing and RMSNorm for $\mathcal{H}^{\text{pre}}$. We avoid materializing either normalized state by applying the corresponding corrections after projection. Let μ and ν denote the mean and second moment of x :

$$Z^{\text{r}} = \frac{xW^{\text{r}} - \mu \mathbf{1}^{\text{T}} W^{\text{r}}}{\sqrt{\nu - \mu^2 + \epsilon}}, \quad Z^{\text{pre}} = \frac{xW^{\text{pre}}}{\sqrt{\nu + \epsilon}}. \quad (20)$$

We concatenate the two weight matrices along the output dimension and compute the raw projections with a single matrix multiplication:

$$[\tilde{Z}^{\text{r}}, \tilde{Z}^{\text{pre}}] = x[W^{\text{r}}, W^{\text{pre}}]. \quad (21)$$

The statistics μ and ν are computed once and shared by both normalization branches. A fused Triton kernel applies the normalization corrections, generates $\mathcal{H}^{\text{pre}}$, and performs fixed-2 plus routed-Top-2 selection. Its backward kernel jointly computes gradients for the normalization corrections and pre-mapping activation, and propagates routing gradients only through the two selected logits.

After routing, a specialized gather kernel extracts $X_{\text{active}} \in \mathbb{R}^{k \times C}$ and computes its flattened-state RMS statistic, avoiding a separate statistics pass. Reusing this statistic, a single active-state projection

jointly generates the post- and residual-mapping logits:

$$[Z^{\text{post}}, Z^{\text{res}}] = \text{RMSNorm}(\text{vec}(X_{\text{active}})) [W^{\text{post}}, W^{\text{res}}]. \quad (22)$$

where $\text{vec}(\cdot)$ flattens the active streams into a row vector. With $k=4$, Sinkhorn normalization operates on a 4×4 residual-mapping matrix per token. The backward kernel recomputes the intermediate Sinkhorn states rather than storing them during the forward pass.

Mapping application. The generated mappings are applied by two fused kernels. Before the Transformer sublayer, the dense pre-mapping read and sparse residual mixing are computed jointly:

$$\text{input} = \sum_{i=1}^N \mathcal{H}_i^{\text{pre}} x_i, \quad X_{\text{mixed}} = \mathcal{H}^{\text{res}} X_{\text{active}}. \quad (23)$$

Without fusion, the dense read $\mathcal{H}^{\text{pre}} X$ and active-stream mixing $\mathcal{H}^{\text{res}} X_{\text{active}}$ would be launched as separate memory-bound kernels. We evaluate both operations in one Triton kernel, reducing kernel-launch overhead. More importantly, the backward kernel accumulates the dense-read and sparse-mixing contributions directly into the gradient of the full residual state, avoiding a separate kC -dimensional active gradient and its subsequent scatter-add.

After the sublayer, a second kernel fuses the post-mapping application, routing-weight scaling, residual addition, and sparse scatter:

$$X_{\mathcal{I}}^{\text{new}} = X_{\text{mixed}} + p \odot (\mathcal{H}^{\text{post}} \text{out}_{\text{aug}}). \quad (24)$$

Fusing post-mapping with residual addition avoids materializing the kC -dimensional write-back update, eliminating one kC write and its subsequent kC read before scatter. We specialize this for the Attention $K_r=1$ path and the MLP $K_r=4$ path with temporal feature augmentation. The backward kernels combine active-gradient gathering with gradients of the sublayer output, post-mapping coefficients, and routing weights, reducing both intermediate tensors and kernel launches.

Temporal feature augmentation. For the MLP path, a specialized kernel jointly evaluates the three causal depthwise-convolution branches with kernel sizes $\{4, 8, 12\}$, reducing kernel launches and allowing overlapping temporal inputs to benefit from cache reuse. This corresponds to C input reads and $3C$ output writes in our memory-access accounting. The subsequent $K_r=4$ post-mapping kernel consumes the original MLP output and the three convolutional outputs directly, avoiding construction of a concatenated $[S, B, 4, C]$ tensor and eliminating an additional $4C$ read and $4C$ write. In backward, a fused input-gradient kernel accumulates contributions from all three branches, while dedicated kernels compute their weight gradients.

Together, these kernels remove normalized full-state intermediates, reuse statistics and fuse both forward and backward mapping application. They reduce implementation overhead beyond the algorithmic memory traffic reported in Table 4; the remaining full-state projection and dense pre-mapping read motivate the cross-sublayer amortization introduced by xHC-Flash in §5.2.

xHC-Flash kernel fusion. For xHC-Flash, we extend the fusion pattern across the two shared sublayers. The router and two sublayer-specific pre-mapping projections are packed into one full-state projection, with normalization corrections, pre-mapping activations, and routed-Top-2 selection applied during the split-dimension reduction. A second full-state pass jointly computes the two dense readouts and gathers the active streams, reusing each loaded state element across these operations. The read-reuse correction is implemented as a small kernel that forms the scalar in Eq. 19 and updates the MLP base readout. Unlike full xHC, where residual mixing is applied before each sublayer, xHC-Flash defers active-stream mixing to the MLP side. This allows the MLP-side kernel to fuse residual mixing, $K_r=4$ post-mapping, routing-weight scaling, and sparse scatter into a single active-state update; backward kernels follow the same fusion boundaries.

End-to-end throughput. We measure wall-clock training throughput on the 18B MoE model without pipeline-communication overlap, to avoid communication-schedule confounders. Our reimplemented mHC ($N=4$) fused kernels add approximately 15% training overhead over the baseline. This is higher than the 6.7% reported in mHC [50], but the results are not directly comparable because the prior measurement may use a different scale, parallelism setup, or overlap schedule. xHC-Flash-4sub adds approximately 11% overhead on top of mHC, mainly from the expanded $N=16$ full-state projection, the dual dense-read pass, and backward residual-stream operations. With pipeline-communication overlap, such as DualPipe [28], the effective end-to-end overhead can be further reduced.

For inference prefill at 2K tokens, mHC adds 11.4% extra overhead over the vanilla baseline, while xHC-Flash-4sub adds 12.9%. This corresponds to only a 1.3% increase in total prefill latency over mHC, aligning with our observation that most of the additional training overhead of xHC-Flash-4sub comes from backward computation rather than the forward residual-stream path.

6 Conclusion

We presented xHC, a method for making residual-stream expansion in HC-family models effective and affordable beyond the common $N=4$ setting. We showed that directly scaling N in mHC is limited by two bottlenecks: insufficient write-back information and the cubic cost of residual-mapping generation. xHC addresses these bottlenecks with temporal feature augmentation and a sparse residual-stream architecture that preserves dense read access while updating only k active streams. Across language model pre-training experiments, xHC scales to $N=16$ with $k=4$, significantly outperforms mHC and the vanilla baseline on 18B and 28B MoE models, improves matched-loss compute efficiency, and remains effective under Muon. xHC-Flash and fused kernels further reduce the memory and implementation overhead needed for practical large- N training. These results support expansion rate as a practical scaling dimension for HC-family models.

References

- [1] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [3] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell, and J. McAuley. Rezero is all you need: Fast convergence at large depth. In *Uncertainty in artificial intelligence*, pages 1352–1361. PMLR, 2021.
- [4] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [6] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [7] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [8] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [9] O. Contributors. Opencompass: A universal evaluation platform for foundation models. <https://github.com/open-compass/opencompass>, 2023.
- [10] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297, 2024.
- [11] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [12] A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, et al. Are we done with mmlu? In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5069–5096, 2025.
- [13] G. H. Golub and C. F. Van Loan. *Matrix computations*. JHU press, 2013.
- [14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [15] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [16] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.
- [17] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

- [18] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [19] T. Henighan, J. Kaplan, M. Katz, M. Chen, C. Hesse, J. Jackson, H. Jun, T. B. Brown, P. Dhariwal, S. Gray, et al. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020.
- [20] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
- [21] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [22] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, Y. Fu, et al. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *Advances in neural information processing systems*, 36:62991–63010, 2023.
- [23] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [24] K. Jordan, Y. Jin, V. Boza, Y. Jiacheng, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon>, 6(3):4, 2024.
- [25] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [26] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- [27] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11260–11285, 2024.
- [28] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [29] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [30] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- [31] Z. Liu, H. Zhang, and A. Li. Beyond the birkhoff polytope: Spectral-sphere-constrained hyper-connections. *arXiv preprint arXiv:2603.20896*, 2026.
- [32] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [33] M. Pagliardini, A. Mohtashami, F. Fleuret, and M. Jaggi. Denseformer: Enhancing information flow in transformers via depth weighted averaging. *Advances in neural information processing systems*, 37:136479–136508, 2024.
- [34] N. Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [35] N. Shazeer. Glue variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

- [36] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [37] R. Sinkhorn and P. Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21(2):343–348, 1967.
- [38] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.
- [39] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [40] K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension. *Transactions of the Association for Computational Linguistics*, 8:141–155, 2020.
- [41] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.
- [42] A. Talmor, J. Herzig, N. Lourie, and J. Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, 2019.
- [43] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [44] K. Team, G. Chen, Y. Zhang, J. Su, W. Xu, S. Pan, Y. Wang, Y. Wang, G. Chen, B. Yin, et al. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.
- [45] H. Touvron, M. Cord, A. Sablayrolles, G. Synnaeve, and H. Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 32–42, 2021.
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [47] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, and F. Wei. Deepnet: Scaling transformers to 1,000 layers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(10):6761–6774, 2024.
- [48] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- [49] F. Wu, A. Fan, A. Baevski, Y. N. Dauphin, and M. Auli. Pay less attention with lightweight and dynamic convolutions. *arXiv preprint arXiv:1901.10430*, 2019.
- [50] Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, et al. mhc: Manifold-constrained hyper-connections. *arXiv preprint arXiv:2512.24880*, 2025.
- [51] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [52] Y. Yang and J. Gao. mhc-lite: You don’t need 20 sinkhorn-knopp iterations. *arXiv preprint arXiv:2601.05732*, 2026.
- [53] Y. Yang, H. Wu, F. Guo, L. Yao, X. Qin, J. Wang, D. Zhang, and J. Yan. Jtok: On token embedding as another axis of scaling law via joint token self-modulation. *arXiv preprint arXiv:2602.00800*, 2026.

- [54] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097, 2025.
- [55] D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyper-connections. *arXiv preprint arXiv:2409.19606*, 2024.
- [56] L. Zhu, Y. Fang, B. Liao, S. Wang, T. Cheng, Z. Huang, C. Chen, L. Wei, Y. Zeng, Y. Wang, et al. Mixture-of-depths attention. *arXiv preprint arXiv:2603.15619*, 2026.

Table 6: Model configurations and training hyperparameters for the main model scales. The 2.5B setting is used for N -sweep experiments, the 10B setting for ablations, and the 18B/28B settings for main results.

Attribute	2.5B	10B	18B	28B
Total Parameters	2.5B	10B	18B	28B
Active Parameters	0.5B	1.4B	1.7B	2.7B
Layers	15	15	28	32
Leading Dense Layers	1	1	1	1
MoE Layers	14	14	27	31
Hidden Dimension	1024	2080	2112	2560
FFN Dimension (dense layer)	3104	6144	6144	6144
Attention Heads	8	16	16	20
KV Groups (GQA)	4	8	8	10
Head Dimension	128	128	128	128
Routed Experts	144	144	144	144
Shared Experts	1	1	1	1
Active Experts (Top- k)	8	8	8	8
Expert FFN Dimension	320	704	672	768
Shared Expert Dimension	320	704	672	768
Normalization	RMSNorm ($\epsilon = 10^{-5}$)			
Position Embedding	RoPE ($\theta = 50000$)			
Activation	SwiGLU			
xHC Expansion Rate N	16	16	16	16
xHC Active Streams k	4	4	4	4
xHC Fixed Streams m	2	2	2	2
Temporal Conv Branches r	3	3	3	3
Conv Kernel Sizes	{4, 8, 12}			
Temporal Conv Initialization	Zero Init			
Gram-Schmidt Orthogonalization	✓			
Temporal Augmentation Position	After MLP (MoE FFN) only			
Stream Router Activation	Sigmoid			
Sinkhorn Iterations	20			
Gating Factor Init α	0.01			
Sequence Length	8192			
Vocab Size	152064			
Tokenizer	Qwen2			
Global Batch Size	160	288	—	768
Training Tokens	—	—	—	—
Warmup Steps	500	500	500	500
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-15}$)			
Base Learning Rate	6.95e-4	4.82e-4	3.97e-4	3.5e-4
LR Schedule	WSD (warmup-stable-decay, exponential decay)			
Min LR Ratio	0.1			
Weight Decay	0.1			
Gradient Clipping	1.0			

A Experimental Details and Hyperparameters

Model Architectures. All MoE backbones follow a DeepSeekMoE-style design: one leading dense layer followed by a stack of MoE layers. Each MoE layer contains routed experts and shared experts with top-8 sigmoid routing. We use grouped-query attention (GQA) with query-key LayerNorm (QK Norm). Detailed configurations are listed in Table 6.

xHC Configuration. Unless otherwise specified, xHC uses $N=16$ total expansion streams with $k=4$ active streams per layer, of which $m=2$ are always-active fixed streams and $k-m=2$ are dynamically selected by the stream router. The router is a single linear projection applied to the LayerNorm-normalized, flattened $N \times C$ residual state, producing N stream scores through a sigmoid activation. Temporal feature augmentation uses $r=3$ causal depthwise convolution branches with kernel sizes {4, 8, 12}, applied only after MLP (MoE FFN) layers, followed by Gram-Schmidt orthogonalization. Sinkhorn normalization for $\mathcal{H}_l^{\text{es}}$ uses 20 iterations, and the gating factor α is initialized to 0.01. During xHC training, we observe that rare extreme activations can hinder the

convergence of Sinkhorn normalization, leaving some row sums greater than one and amplifying the forward signal. To improve stability, we apply row-sum clamping after Sinkhorn normalization by replacing each row $\mathcal{H}_{l,i}^{\text{res}}$ with $\mathcal{H}_{l,i}^{\text{res}} / \max(\sum_j \mathcal{H}_{l,ij}^{\text{res}}, 1)$. This operation only rescales rows whose sums exceed one, and we observe that it stabilizes training without degrading the performance.

***N*-sweep Configuration.** The *N*-sweep experiments in Section 4.4 use the 2.5B setting in Table 6. Table 7 lists the xHC active-stream configuration for each expansion rate. For mHC, we sweep dense expansion rates $N \in \{2, 4, 8, 16\}$, where all *N* streams participate in residual mixing and write-back. Other training settings, including tokenizer, context length, data mixture, optimizer, learning-rate schedule, and training-token count, are kept matched across compared runs.

Table 7: *N*-sweep configuration for xHC.

Expansion rate <i>N</i>	Active streams <i>k</i>	Fixed streams <i>m</i>
2	1	0
4	2	1
8	4	2
16	4	2

Training Hyperparameters. At each model scale, xHC, mHC, and the vanilla residual baseline are trained with matched optimization recipes and data budgets, so that differences can be attributed to the residual-stream design rather than the training setup. We use AdamW with $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-15}$, weight decay 0.1, and gradient clipping at 1.0. The learning rate follows a warmup–stable–decay (WSD) schedule with 500 warmup steps, exponential decay, and a minimum-to-peak learning-rate ratio of 0.1. The base learning rate and global batch size are scaled with model size, as listed in Table 6.

Final Stream Reduction. For both mHC and xHC, the multi-stream hidden state is reduced back to a single hidden vector before the language-model head. Specifically, before applying the final RMSNorm and unembedding layer, we sum the *N* residual streams within each token and use the resulting *C*-dimensional representation for prediction.

Muon Optimizer Configuration. For the experiments in Section 4.6, we use a hybrid setup: Muon is applied to backbone 2D weight matrices (attention, MLP/MoE, and MoE router projections), while AdamW is used for other parameters (embeddings, normalization, and all xHC-specific parameters). Muon uses momentum $\beta=0.95$, 5 Newton–Schulz iterations, and a matched-AdamW-RMS target of 0.2. Other hyperparameters and the model architecture follow the 18B setting in Table 6.

Evaluation Protocol. All benchmarks are evaluated under the modified OpenCompass framework [9]. We use two evaluation paradigms: 1) *Perplexity-based evaluation*. The model scores each candidate answer by computing its conditional log-likelihood given the prompt and few-shot exemplars; the option with the highest likelihood is selected. This is used for multiple-choice benchmarks where answer options are fixed: MMLU (5-shot), MMLU-Redux (5-shot), CMMLU (5-shot), CEval (5-shot), ARC-Challenge (25-shot), C3 (3-shot). 2) *Generation-based evaluation*. The model generates a free-form response, which is then parsed and matched against the reference answer (exact match or rule-based extraction). This is used for: MMLU-Pro (5-shot), BBH (3-shot), CommonsenseQA (7-shot), GSM8K (4-shot), HumanEval (0-shot, pass@1), and LCBench (5-shot).

B Scaling Law Experiment Configuration

We train four model sizes for each method (vanilla baseline, mHC, and xHC) under matched recipes, spanning training compute from approximately 1.7×10^{19} to 4.0×10^{20} FLOPs. Table 8 lists the key model configurations used for scaling-law fitting. All models follow the same DeepSeekMoE-style template as Table 6, with 144 routed experts, top-8 routing, one shared expert, and one leading dense layer, while varying model width, depth, and expert FFN size. Scaling laws are fitted on the final language modeling (LM) loss.

We fit a shifted power law $\mathcal{L}(C) = AC^{-\alpha} + E$ to the final LM loss of each method, where *C* denotes training FLOPs and $E = 0.72$ is an estimated irreducible loss. Following our fitting code, we perform linear regression on $\log_2(\mathcal{L} - E)$ against $\log_{10} C$ and convert the resulting fit back to the shifted power-law form above. The fitted parameters are reported in Table 9.

Table 8: Scaling-law model configurations. All models use context length 8192, RoPE [39] ($\theta=50000$), 144 routed experts, top-8 routing, 500 warmup steps, and the same data mixture.

Active Parameters [†]	Total Parameters	Layers	MoE Layers	Hidden Dimension	FFN Dimension (dense layer)	Expert FFN Dimension	Q/KV Heads	Training Budget	Global Batch Size	Base Learning Rate
180M	1.08B	16	15	656	1920	224	8/4	–	128	8.28e-4
340M	2.20B	15	14	1024	3104	320	8/4	–	160	6.95e-4
460M	3.74B	23	22	1040	3072	352	8/4	–	208	5.74e-4
1.10B	9.46B	15	14	2080	6144	704	16/8	–	288	4.82e-4

[†] Activated parameters per token, excluding embeddings.

Table 9: Fitted scaling-law parameters.

Method	A	α
Baseline	109.303	0.0936
mHC ($N=4$)	99.139	0.0920
xHC ($N=16, k=4$)	97.703	0.0919

C Training FLOPs and Parameter Overhead Analysis

We derive the per-layer parameter overhead introduced by xHC and mHC and report the resulting training FLOPs cost.

C.1 Per-Layer Parameter Overhead

Each Transformer layer contains two sublayers (attention and MLP). Let C denote the hidden dimension, N the expansion rate, k the number of active streams, K_r the temporal augmentation rank, and $\{\kappa_1, \dots, \kappa_r\}$ the depthwise convolution kernel sizes. Temporal augmentation is applied only after the MLP sublayer.

xHC. Each sublayer adds a stream router (N^2C), a pre-mapping generator (N^2C), and a residual-mapping generator (k^3C). The post-mapping generator operates at rank K_r for the MLP sublayer (with temporal augmentation) and rank 1 for the attention sublayer. Depthwise convolution weights are added only to the MLP sublayer. The total per-layer count is:

$$P_{\text{xHC}} = (4N^2 + 2k^3 + k^2 + k^2K_r + \sum_{i=1}^r \kappa_i) \cdot C. \quad (25)$$

mHC. All N streams are active ($k=N$), with no router, no temporal augmentation ($K_r=1$), and no convolution:

$$P_{\text{mHC}} = (4N^2 + 2N^3) \cdot C. \quad (26)$$

The $2N^3C$ residual-mapping term is the source of cubic cost scaling.

Comparison. With $K_r=4$ and kernel sizes $\{4, 8, 12\}$, xHC has the following per-layer overheads under representative active-stream budgets:

$$\begin{aligned} P_{\text{xHC}, N=4, k=2} &= 124C, & P_{\text{xHC}, N=16, k=4} &= 1256C, \\ P_{\text{mHC}, N=4} &= 192C, & P_{\text{mHC}, N=16} &= 9216C. \end{aligned}$$

xHC at $N=16, k=4$ uses **7.3 $\times$ fewer** parameters per layer than dense mHC at the same expansion rate, while providing the same $N=16$ residual-memory capacity.

C.2 Training FLOPs Overhead

The parametric training FLOPs overhead per token introduced by the residual-stream module is:

$$F_{\text{HC}} = 6 \cdot P_{\text{HC}} \cdot L, \quad (27)$$

where P_{HC} is instantiated as P_{xHC} or P_{mHC} depending on the method. The factor of 6 accounts for forward and backward passes through parametric projections. Non-parametric operations such as Sinkhorn normalization, residual mixing, LayerNorm on the multi-stream state, and Gram–Schmidt projections are analyzed separately in the memory-access and infrastructure discussions.

We compute the backbone training FLOPs per token F_{backbone} using the standard $6\times$ approximation for linear layers, plus the sequence-dependent attention cost $6Sn_qd_hL$ (assuming causal attention with average context length $S/2$) and the LM-head projection $6VC$ [25]. The overhead percentage is $\Delta\% = F_{\text{HC}}/F_{\text{backbone}} \times 100$. Table 10 reports the overhead across methods and scales.

Table 10: Parameter and training FLOPs overhead. P/layer is the per-layer HC parameter count; $\Delta P\%$ is the parameter overhead relative to the vanilla backbone’s activated parameters; $\Delta F\%$ is the training FLOPs overhead.

	18B ($C=2112, L=28$)			28B ($C=2560, L=32$)		
	P/layer	$\Delta P\%$	$\Delta F\%$	P/layer	$\Delta P\%$	$\Delta F\%$
mHC ($N=4$)	405K (192C)	0.5%	0.7%	492K (192C)	0.6%	0.5%
mHC ($N=16$)	19.5M (9216C)	26.3%	18.9%	23.6M (9216C)	30.2%	22.3%
xHC ($N=16, k=4$)	2.65M (1256C)	3.5%	4.1%	3.22M (1256C)	4.1%	3.0%

xHC at $N=16$ incurs only modest additional training FLOPs across the main model scales, while dense mHC at the same expansion rate requires $\sim 19\%$ overhead at 18B. For xHC, the overhead percentage decreases with model scale because F_{backbone} grows with width while P_{xHC} scales only linearly in C . The parameter overhead follows the same trend, confirming that the sparse architecture keeps the cost of large- N expansion modest across scales.

D Additional Ablation Studies

We provide additional ablations on temporal feature augmentation, covering where augmentation is applied, whether Gram–Schmidt orthogonalization affects performance, and whether multi-scale temporal components are beneficial. Unless otherwise specified, experiments are conducted under the same 10B MoE setting as Section 4.5, and we report validation loss on the Pile test set.

Table 11: Additional ablations on temporal feature augmentation. Lower validation loss is better.

Variant	Val. Loss $\downarrow$
xHC default	1.983
w/o Gram–Schmidt orthogonalization	1.984
+ Attention-side temporal augmentation	1.985

Attention-side Temporal Feature Augmentation. In xHC, temporal feature augmentation is applied only after MLP (MoE FFN) layers rather than after attention layers. This design follows the different roles of the two sublayers: attention already performs content-dependent information mixing across tokens, whereas the MLP sublayer processes each token independently and therefore provides a cleaner place to inject lightweight local contextual features. We also test adding the same temporal convolution after attention layers. As shown in Table 11, this variant gives a similar validation loss but is slightly worse than the default design by 0.002, indicating that attention-side temporal augmentation brings little additional benefit while introducing extra computation and implementation complexity. We therefore use temporal augmentation only after MLP layers throughout the paper.

Effect of Gram–Schmidt Orthogonalization. Section 3.3.1 argues that convolutional components can retain strong components aligned with the original layer output. Directly combining these correlated components may amplify the original write-back direction and make write-back scaling less predictable. We therefore apply Gram–Schmidt orthogonalization to remove collinear components and keep the augmented write-back components distinct. As shown in Table 11, removing Gram–Schmidt orthogonalization gives nearly the same validation loss as the default xHC variant at the 10B ablation scale, indicating that this normalization does not remove useful signal. However, at the 18B scale, the cosine similarity between convolutional branches and the main branch can exceed 0.7, and removing Gram–Schmidt orthogonalization leads to training instability. We therefore retain Gram–Schmidt orthogonalization as a low-cost conditioning step that improves large-scale training stability while preserving validation performance.

Effect of multi-scale temporal components. We ablate the number of temporal convolution branches used in temporal feature augmentation. This experiment is conducted on dense mHC at

Table 12: Ablation on the number of temporal convolution branches. All variants use dense mHC at $N=16$ without sparse residual updates.

Variant	Conv Branches	Val. Loss ↓
mHC	0	1.998
mHC + Temp Aug (single-scale)	1	1.989
mHC + Temp Aug (multi-scale)	3	1.984

$N=16$, without the sparse residual-stream architecture, so that the effect of enriching the write-back signal can be isolated. As shown in Table 12, adding one causal convolution branch improves validation loss from 1.998 to 1.989, confirming that local temporal information provides useful additional write-back signal. Using three branches with different kernel sizes further improves the loss to 1.984. This supports our design choice: multiple convolution branches provide write-back components over different temporal ranges, making the augmented signal more diverse and more effective for large- N expansion.

E Design Analysis of xHC-Flash

xHC-Flash removes Attention-side residual mixing so that later sublayer inputs can be recovered exactly from precomputed base readouts and the sparse write-back updates, without rereading the full N -stream state. We first derive the correction for the two-sublayer variant and then generalize it to xHC-Flash-4sub.

E.1 Two-Sublayer Dense-Read Reuse

Let $X^{(0)} = (x_1^{(0)}, \dots, x_N^{(0)})^\top$ denote the block-entry state, and let $\mathcal{I} = (\mathcal{I}_1, \dots, \mathcal{I}_k)$ denote the active-stream indices. The MLP pre-mapping is generated from $X^{(0)}$, and its base readout is precomputed as

$$\text{inp}_M = \sum_{i=1}^N \mathcal{H}_i^{\text{pre,MLP}} x_i^{(0)}. \quad (28)$$

Because the Attention sublayer does not apply residual mixing, its write-back changes active stream $\mathcal{I}_j$ by

$$x_{\mathcal{I}_j}^{(A)} = x_{\mathcal{I}_j}^{(0)} + p_j \mathcal{H}_j^{\text{post,Attn}} \text{out}_{\text{Attn}}, \quad j = 1, \dots, k, \quad (29)$$

while inactive streams remain unchanged. Applying the fixed MLP pre-mapping to the post-Attention state gives

$$\text{input}_{\text{MLP}} = \text{inp}_M + \sum_{j=1}^k \mathcal{H}_{\mathcal{I}_j}^{\text{pre,MLP}} p_j \mathcal{H}_j^{\text{post,Attn}} \text{out}_{\text{Attn}} \quad (30)$$

$$= \text{inp}_M + \alpha \text{out}_{\text{Attn}}, \quad (31)$$

where

$$\alpha = \sum_{j=1}^k \mathcal{H}_{\mathcal{I}_j}^{\text{pre,MLP}} p_j \mathcal{H}_j^{\text{post,Attn}}. \quad (32)$$

Thus, α is a token-wise scalar assembled from dynamically generated mapping coefficients, and Eq. 19 requires only a scalar-vector multiply and an addition after the base readout has been computed.

E.2 Four-Sublayer Extension

xHC-Flash-4sub jointly forms four separate pre-mappings $\{\mathcal{H}^{\text{pre},t}\}_{t=1}^4$ from the group-entry state, while sharing one routing decision $(\mathcal{I}, p)$ across the group. Unlike the two-sublayer case, later sublayers must consume both Attention and MLP write-backs; the MLP write-back has $K_r=4$ temporal feature components and therefore cannot always be reduced to a scalar multiple of a single

output vector. We therefore express later inputs using a non-active base readout plus the current active-stream state, avoiding a separate accumulated-delta buffer.

For sublayer q , let K_q denote the number of write-back components and let $\text{out}_{\text{aug},r}^{(q)}$ denote component r . Its sparse write-back to active stream $\mathcal{I}_j$ is

$$\Delta x_{\mathcal{I}_j}^{(q)} = p_j \sum_{r=1}^{K_q} \mathcal{H}_{j,r}^{\text{post},q} \text{out}_{\text{aug},r}^{(q)}. \quad (33)$$

In the two-sublayer xHC-Flash case, the only preceding write-back before the MLP input is the Attention write-back with $K_q=1$. The active-stream correction therefore factorizes into one scalar coefficient multiplying out_{Attn} , yielding Eq. 19. For any preceding write-back q , the active correction takes the form

$$\sum_{j=1}^k \mathcal{H}_{\mathcal{I}_j}^{\text{pre},t} \Delta x_{\mathcal{I}_j}^{(q)} = \sum_{r=1}^{K_q} \left(\sum_{j=1}^k \mathcal{H}_{\mathcal{I}_j}^{\text{pre},t} p_j \mathcal{H}_{j,r}^{\text{post},q} \right) \text{out}_{\text{aug},r}^{(q)}. \quad (34)$$

When q is an MLP sublayer, $K_q=K_r=4$, so this term is a weighted sum over multiple augmented output vectors rather than a scalar multiple of a single vector. A direct implementation could maintain these accumulated active-stream deltas explicitly, but doing so would introduce an extra $[S, B, k, C]$ buffer, including buffer updates after write-back and a later active-delta read for the correction.

Instead, for later sublayers we decompose the dense readout into a non-active base readout and the contribution of the current active streams:

$$\text{base}_{-\mathcal{I}}^{(t)} = \sum_{i \notin \mathcal{I}} \mathcal{H}_i^{\text{pre},t} x_i^{(0)}, \quad t = 1, \dots, 4. \quad (35)$$

Let $x_{\mathcal{I}_j}^{(t)}$ denote the current active-stream state before sublayer t . The input to sublayer t is then

$$\text{inp}_{\text{act}}^{(t)} = \sum_{j=1}^k \mathcal{H}_{\mathcal{I}_j}^{\text{pre},t} x_{\mathcal{I}_j}^{(t)}. \quad (36)$$

$$\text{input}^{(t)} = \text{base}_{-\mathcal{I}}^{(t)} + \text{inp}_{\text{act}}^{(t)}. \quad (37)$$

Because residual mixing is deferred to the final MLP, intermediate sublayers update only the selected active streams, so

$$x_{\mathcal{I}_j}^{(t)} = x_{\mathcal{I}_j}^{(0)} + \sum_{q < t} \Delta x_{\mathcal{I}_j}^{(q)}. \quad (38)$$

Substituting Eq. 38 into Eq. 36 decomposes the active contribution into its group-entry value and the accumulated sparse write-backs. Together with the non-active base in Eq. 35, this recovers the same input that would be obtained by applying all preceding sparse write-backs to the group-entry readout, but without explicitly storing them.

Implementation note. The derivation above uses the non-active-base form because it makes the input correction and memory accounting explicit: later sublayer inputs are formed from the unchanged non-active streams and the current active streams, without materializing a separate delta buffer. Our fused implementation follows the same correction mathematically, but may schedule the equivalent operations differently to improve kernel efficiency. These implementation choices do not change the resulting sublayer inputs.

E.3 Exactness, Complexity, and Design Trade-offs

This subsection clarifies what is exact in xHC-Flash and what is approximated relative to full xHC. The input corrections above are exact provided that (i) the routing decision and sublayer-specific pre-mappings are fixed from the sharing-window entry state, (ii) residual mixing is not applied before an intermediate sublayer, and (iii) each sparse write-back modifies only the selected active streams. Residual mixing at the final MLP does not violate these conditions because its input has already been formed and no subsequent sublayer in the sharing group requires another correction.

Under these conditions, the correction recovers the same sublayer input that would be obtained by applying the fixed pre-mapping to the updated residual state, without an $O(NC)$ dense read

over all streams. For the two-sublayer case, this reduces to the scalar–vector form in Eq. 19. For xHC-Flash-4sub, the correction uses the active-state form in Eq. 37, relying on the active streams already carried through the sharing group. If residual mixing were applied before an intermediate sublayer, the correction would additionally depend on the mixed active state and would no longer admit this simple active-state form.

Thus, the approximation in xHC-Flash is not the dense-read correction itself. Relative to full xHC, xHC-Flash approximates the dynamic stream-control schedule: routing is shared over the local window, sublayer-specific pre-mappings are generated from the window-entry state rather than from each updated intermediate state, and residual mixing is deferred to the final MLP in the sharing group. These choices are motivated by the sparse update structure: only k of the N streams change at each sublayer, while the exact correction exposes those changes to subsequent sublayers.

Why the approximation is mild. The sharing window is short: one Transformer block for xHC-Flash and two blocks for xHC-Flash-4sub. Empirically, mHC at $N=4$ already achieves substantial gains with a much smaller stream-control space, suggesting that HC-family models do not require fully independent control at every sublayer to be effective. xHC-Flash also retains the main xHC mechanisms, including dense read access, separate sublayer-specific pre-mapping weights, sparse write-back to active streams, and MLP-side residual mixing. These factors help explain why the measured validation loss remains close to full xHC despite substantially lower memory traffic, with 1.983 for both full xHC and xHC-Flash, and 1.984 for xHC-Flash-4sub (Table 5).